

NovaCart Order Analytics: Submission Deliverables

Team Bug Byts · Microsoft Fabric workspace NovaCart_HCL · pipeline novacart_medallion · exported from Fabric on 6 Oct 2026 · repo github.com/sayantansikdar/novacart-medallion-pipeline

Contents: **1** pipeline definition (export) · **2** monitored runs (batch 1, batch 2, batch 2 again, deliberate failure) · **3** data-quality report per batch · **4** table reliability evidence (Part 5) and extras (Part 6). Raw exports are in *deliverables/raw/* and *evidence/run_logs/*; screenshots of the same runs: Fabric → Monitor → novacart_medallion → View run details.

1 · Pipeline definition (exported from Fabric)

Parameter: **batch_id** (string, default '1'). Every notebook step receives *batch_id* = `@pipeline().parameters.batch_id`.

Step	Type	Notebook	Runs after (condition)	Retries	Retry interval	Timeout
setup	TridentNotebook	00_setup	start	2	60 s	0.01:00:00
bronze	TridentNotebook	01_bronze	setup (Succeeded)	2	60 s	0.01:00:00
silver_reference	TridentNotebook	03_silver_reference	bronze (Succeeded)	2	60 s	0.01:00:00
silver_orders	TridentNotebook	02_silver_orders	silver_reference (Succeeded)	2	60 s	0.01:00:00
silver_items	TridentNotebook	04_silver_items	silver_orders (Succeeded)	2	60 s	0.01:00:00
gold	TridentNotebook	05_gold	silver_items (Succeeded)	2	60 s	0.01:00:00
notify_failure	TridentNotebook	99_notify_failure	gold (Failed or Skipped)	1	60 s	0.01:00:00
fail_run	Fail	—	notify_failure (Succeeded)	—	—	—

Flow: setup → bronze → silver_reference → silver_orders → silver_items → gold. If any step fails (after its retries) the steps after it are skipped, so *gold* ends Failed or Skipped; that starts **notify_failure** (records the alert in `silver.pipeline_alerts` and an alert file), then **fail_run** marks the whole run Failed in Monitor. Full JSON: *deliverables/raw/pipeline_definition.json*.

2 · Monitored runs

Run	Run id	Status	Start (UTC)	End (UTC)	Duration
batch_id = abc (deliberate failure)	cf7e38b3...	Failed	2026-10-06 11:30:19	2026-10-06 11:35:42	5.4 min
batch 1	afdf5e87...	Completed	2026-10-06 11:41:26	2026-10-06 11:54:38	13.2 min
batch 2	0a7efe86...	Completed	2026-10-06 11:55:01	2026-10-06 12:11:13	16.2 min
batch 2 (again)	c12f4674...	Completed	2026-10-06 12:11:24	2026-10-06 12:27:49	16.4 min

Run: batch_id = abc (deliberate failure) — Failed

Step	Status	Start (UTC)	Duration	Attempt	Error
setup	Failed	2026-10-06 11:30:30	29 s	1	Notebook execution failed at Notebook service with http status code - '200', please check the Run logs on Note
setup	Failed	2026-10-06 11:32:00	20 s	2	Notebook execution failed at Notebook service with http status code - '200', please check the Run logs on Note
setup	Failed	2026-10-06 11:33:22	20 s	3	Notebook execution failed at Notebook service with http status code - '200', please check the Run logs on Note
notify_failure	Succeeded	2026-10-06 11:33:47	111 s	1	
fail_run	Failed	2026-10-06 11:35:40	0 s	1	NovaCart pipeline failed; see the failed step in Monitor. An alert was recorded by notify_failure.

Run: batch 1 — Completed

Step	Status	Start (UTC)	Duration	Attempt	Error
setup	Succeeded	2026-10-06 11:41:33	103 s	1	
bronze	Succeeded	2026-10-06 11:43:17	111 s	1	
silver_reference	Succeeded	2026-10-06 11:45:11	218 s	1	
silver_orders	Succeeded	2026-10-06 11:48:51	98 s	1	
silver_items	Succeeded	2026-10-06 11:50:32	96 s	1	
gold	Succeeded	2026-10-06 11:52:11	142 s	1	

Run: batch 2 — Completed

Step	Status	Start (UTC)	Duration	Attempt	Error
setup	Succeeded	2026-10-06 11:55:08	50 s	1	
bronze	Succeeded	2026-10-06 11:56:00	232 s	1	
silver_reference	Succeeded	2026-10-06 11:59:54	156 s	1	
silver_orders	Succeeded	2026-10-06 12:02:33	157 s	1	
silver_items	Succeeded	2026-10-06 12:05:12	141 s	1	
gold	Succeeded	2026-10-06 12:07:34	216 s	1	

Run: batch 2 (again) — Completed

Step	Status	Start (UTC)	Duration	Attempt	Error
setup	Succeeded	2026-10-06 12:11:31	50 s	1	
bronze	Succeeded	2026-10-06 12:12:22	217 s	1	
silver_reference	Succeeded	2026-10-06 12:16:00	172 s	1	
silver_orders	Succeeded	2026-10-06 12:18:54	156 s	1	
silver_items	Succeeded	2026-10-06 12:21:32	155 s	1	
gold	Succeeded	2026-10-06 12:24:08	217 s	1	

The failed run shows the on-failure path working: *setup* rejected batch_id 'abc' three times (first attempt + 2 retries, about 60 s apart), every later step was skipped (not started), *notify_failure* recorded the alert and *fail_run* marked the run Failed.

Recorded alert: 2026-10-06T11:35:14+00:00 — NovaCart pipeline run cf7e38b3-a157-4203-9a7a-eb489fac3f7c failed for batch_id='abc'. See Monitor > b040c72b-79c5-4d70-bf44-bd7a77bbdd33 for the failed step and its error.

3 · Data-quality report

Every step writes row counts to silver.dq_report and refuses to write unless **rows in = rows out + quarantined + deduplicated**. Each gold run exports its batch's report as JSON (*deliverables/raw/dq_batch_N.json*). Inserted/updated are the MERGE results in silver.

Batch 1

Layer	Entity	Rows in	Rows out	Quarantined	Deduplicated	Inserted	Updated	Balanced
bronze	customers_changes	194	194	0	0			PASS
bronze	fx_rates	84	84	0	0			PASS
bronze	order_items	1006	1006	0	0			PASS
bronze	orders	1418	1418	0	0			PASS
bronze	payments	1164	1164	0	0			PASS
bronze	products	40	40	0	0			PASS
silver	dim_customer	194	166	5	23	166	0	PASS
silver	fx_rates	84	84	0	0	84	0	PASS
silver	order_items	1006	971	10	25	971	0	PASS
silver	orders	1418	495	0	923	495	0	PASS
silver	payments	1164	1164	0	0	1164	0	PASS
silver	products	40	40	0	0	40	0	PASS
gold	fact_order_line	971	971	0	0			PASS
Quarantine rule		Layer / entity			Rows quarantined			
UPDATED_AT_MISSING		silver / dim_customer			5			
ORDER_NOT_FOUND		silver / order_items			10			
Reconciliation check				Value	Expected	Result		
fact lines = silver order_items lines				971	971	PASS		
daily_revenue total = fact revenue total				150062.6435	150062.6435	PASS		
revenue_by_category total = fact revenue total				150062.6435	150062.6435	PASS		
lines without an FX rate (fx_missing, excluded)				0	0	info		
lines for customers not in the CRM (customer_sk = -1)				18	—	info		
payments whose order is unknown (excluded from mismatches)				537	—	info		
orders with a payment mismatch > 0.50 USD				64	—	info		

Batch 2

Layer	Entity	Rows in	Rows out	Quarantined	Deduplicated	Inserted	Updated	Balanced
bronze	customers_changes	194	194	0	0			PASS
bronze	fx_rates	84	84	0	0			PASS
bronze	order_items	1007	1007	0	0			PASS
bronze	orders	1545	1545	0	0			PASS
bronze	payments	1164	1164	0	0			PASS
bronze	products	40	40	0	0			PASS
silver	dim_customer	194	166	5	23	0	0	PASS
silver	fx_rates	84	84	0	0	0	0	PASS
silver	order_items	1007	974	8	25	0	0	PASS
silver	orders	1545	597	0	948	0	0	PASS
silver	payments	1164	1164	0	0	0	0	PASS
silver	products	40	40	0	0	0	0	PASS
gold	fact_order_line	1933	1933	0	0			PASS
Quarantine rule		Layer / entity			Rows quarantined			
UPDATED_AT_MISSING		silver / dim_customer			5			
ORDER_NOT_FOUND		silver / order_items			8			
Reconciliation check				Value	Expected	Result		
fact lines = silver order_items lines				1933	1933	PASS		
daily_revenue total = fact revenue total				295585.5703	295585.5703	PASS		
revenue_by_category total = fact revenue total				295585.5703	295585.5703	PASS		
lines without an FX rate (fx_missing, excluded)				0	0	info		
lines for customers not in the CRM (customer_sk = -1)				39	—	info		
payments whose order is unknown (excluded from mismatches)				5	—	info		
orders with a payment mismatch > 0.50 USD				13	—	info		

Batch 2 shows 0 inserted / 0 updated because each batch's DQ rows are replaced on every run and the last run of batch 2 was the deliberate re-run, which changed nothing. The first batch 2 run inserted 450 orders and updated 137 (silver.orders history, version 4) and inserted 962 order lines.

Reading the numbers. Snapshot files (customers, products, FX, payments) are reloaded in every batch, so batch 2 shows them again with 0 inserted / 0 updated (nothing changed). Deduplicated = exact duplicates and older versions within the batch; for dim_customer it also counts CRM rows that start no new SCD2 version (e-mail/name-only changes). The 5 CRM rows without updated_at and the order lines whose order

does not exist (NC-990xxx) are the quarantined rows. After batch 1 the payments snapshot already contains the whole month, so many payments point at orders that only arrive in batch 2; they are counted, not compared.

4 · Table reliability evidence (Part 5) and extras (Part 6)

Produced by the notebook 06_evidence on the official tables; 11 of 11 automatic checks passed. Full write-up: evidence/EVIDENCE.md.

5.1 Change history of silver.orders (batch 1, batch 2, batch 2 again)

Version	Timestamp (UTC)	Operation	userMetadata	Inserted	Updated
0	2026-10-06 11:42:01	CREATE TABLE	step=setup		
1	2026-10-06 11:42:11	ADD CONSTRAINT	step=setup		
2	2026-10-06 11:42:16	ADD CONSTRAINT	step=setup		
3	2026-10-06 11:50:07	MERGE	batch_id=1 step=silver.orders	495	0
4	2026-10-06 12:04:57	MERGE	batch_id=2 step=silver.orders	450	137

v3 = batch 1, v4 = batch 2. **Batch 2 run again produced no new version:** the MERGE found nothing newer, and Delta writes no commit for a MERGE that changes nothing (the re-run is in the run list above and in the DQ report with 0 inserted, 0 updated).

5.2 Point-in-time read (state right after batch 1)

SELECT * FROM LH_NovaCart_Silver.dbo.orders VERSION AS OF 3 → **495 orders** (now 945); same result with TIMESTAMP AS OF '2026-10-06 11:50:07.206000'. Status mix then: cancelled 59, delivered 274, paid 45, placed 51, shipped 66.

5.3 Schema enforcement

Attempt: append 1 row of silver.orders with an extra column loyalty_points. **Result:** rejected — [LEGACY_ERROR_TEMP_DELTA_0007] A schema mismatch detected when writing to the Delta table (Table ID: 68dbd529-1650-4581-9274-b45dc507c19f). Table version 4 → 4 (unchanged).

What happened: Delta keeps the table schema in its transaction log and validates every write against it; a DataFrame with an unknown column is refused as a whole before anything is committed. Adding the column would need an explicit mergeSchema opt-in, which silver never uses (only bronze does, so new source columns are kept there for review). **CHECK constraint:** insert an order with status 'refunded' → : org.apache.spark.sql.delta.schema.DeltaInvariantViolationException: [DELTA_VIOLATE_CONSTRAINT_WITH_VALUES] CHECK constraint orders_status_valid (status IN ('placed', 'paid', 'shipped', 'delivered', 'cancelled')) violat

5.4 Storage optimisation on the fact table

	Files	Size (bytes)	Rows	Sum of net_usd
Before (this run)	1	83705	1933	351746.0626
After (this run)	1	83705	1933	351746.0626

OPTIMIZE commits in the table history: v3 (2026-10-06 12:46:29): **13 files** → **1**; v4 (2026-10-06 12:50:48): **1 files** → **1**; v5 (2026-10-06 12:56:38): **1 files** → **1**; v6 (2026-10-06 13:02:46): **1 files** → **1** (the first one compacted the files written by the gold rebuild; later runs had nothing left to compact).

OPTIMIZE LH_NovaCart_Gold.dbo.fact_order_line ZORDER BY (order_date): same data, fewer files. **Partition** only large tables by a low-cardinality column every query filters on (fact at scale: order_date/month, ~1 GB per partition); this data is a few hundred KB, so partitions would just create tiny files. Use **Z-order/clustering** for selective high-cardinality filters (customer_id, product_id); Fabric's V-Order sorts and compresses for fast reads. **VACUUM warning:** on a throwaway copy, after VACUUM RETAIN 0 HOURS reading version 0 failed (An error occurred while calling o9766.collectToPython.): VACUUM permanently removes the files old versions need, which would break the point-in-time read and change feed above. Keep the default 7-day retention or longer.

6.1 Row-level changes in batch 2 (Change Data Feed on silver.orders)

Version 4: insert 450, update_postimage 137, update_preimage 137.

Status before	Status after	Orders
shipped	delivered	66
paid	delivered	43
placed	delivered	26
paid	cancelled	2

6.2 Incremental gold

From gold as of batch 1, only the **608** orders touched by batch 2 were rebuilt (**1266** of 1933 lines): **0 differences** from the full rebuild. Valid while reference tables are unchanged in the batch (changed: none); otherwise rebuild in full.

6.3 Automatic ingestion

A Fabric storage-event trigger on the landing folder (landing/batch=N/) starts the pipeline with batch_id from the folder name as soon as files arrive, with a daily schedule as a safety net (re-runs are harmless). For many small files, Structured Streaming with a checkpoint and trigger(availableNow=True) processes each new file exactly once (Fabric's equivalent of Auto Loader). **Trigger choice:** fixed schedule = simple but batch 2's arrival time is unknown; windowed schedule = good for regular time slices with backfill, still polls; **file-arrival event = chosen**, runs exactly when a batch lands.

All checks

Result	Check
PASS	R1 one MERGE commit per batch, tagged in userMetadata
PASS	R1 the batch 2 re-run made no commit (no rows changed, so no new version)
PASS	R2 VERSION AS OF 3 returns the post-batch-1 table (495 orders)
PASS	R2 TIMESTAMP AS OF the batch-1 commit returns the same rows
PASS	R3 append with an extra column is rejected; table version unchanged
PASS	R4 status 'refunded' violates the CHECK constraint; version unchanged
PASS	R5 OPTIMIZE compacted the fact table's files; data identical

Result	Check
PASS	VACUUM removes the files of old versions: time travel to them fails afterwards
PASS	CDF: batch 2 = 450 inserted orders and 137 updated orders
PASS	reference tables unchanged in batch 2, so an incremental update is valid
PASS	incremental gold (only changed orders rebuilt) = full rebuild